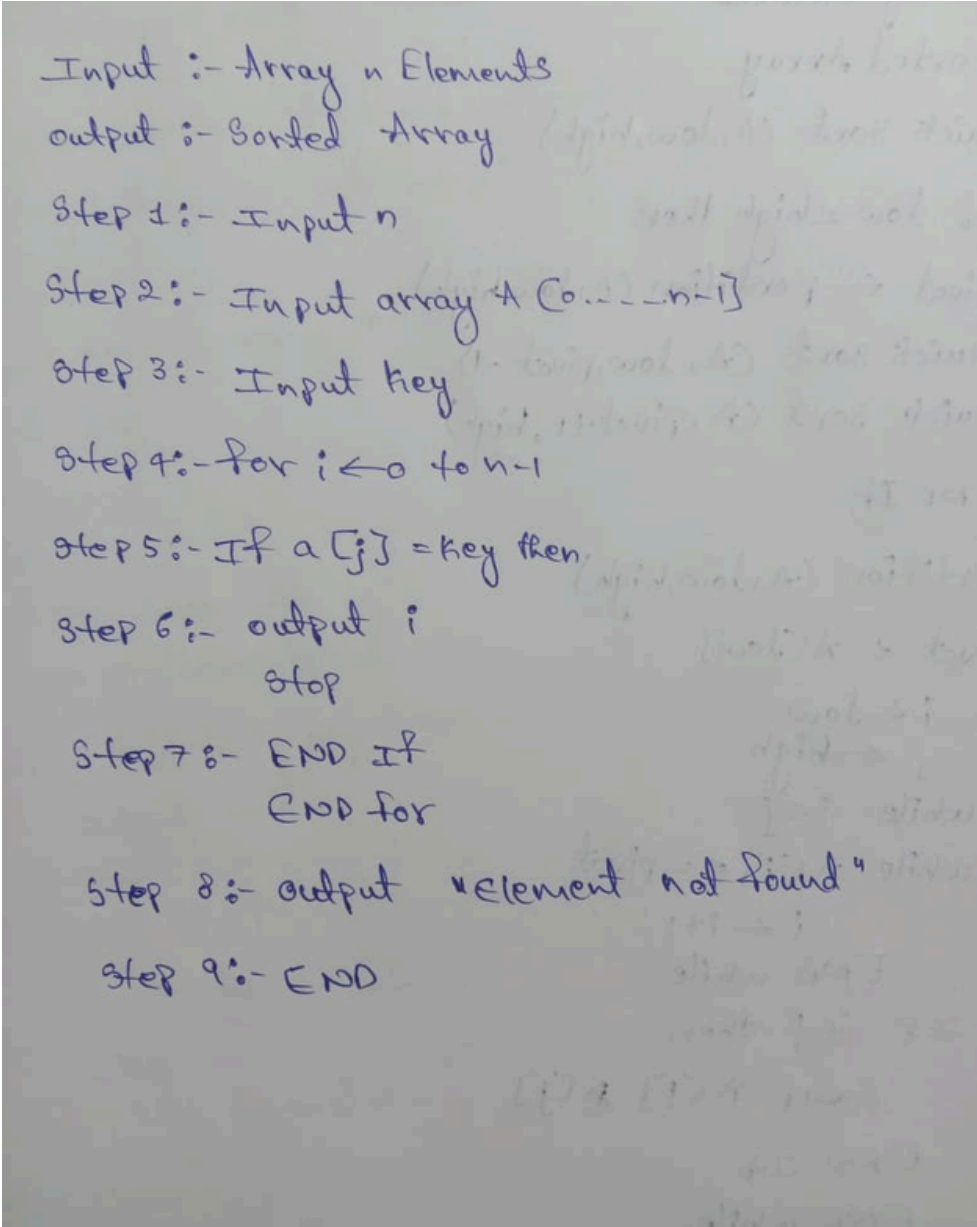


Practical-2

Aim: Implementation and Time analysis of linear and binary search algorithm.

1) Linear search



```
Input :- Array n Elements
output :- Sorted Array
Step 1:- Input n
Step 2:- Input array A [0...n-1]
Step 3:- Input key
Step 4:- for i ← 0 to n-1
Step 5:- If a[i] = key then
Step 6:- output i
           stop
Step 7:- END IF
           END for
Step 8:- output "element not found"
Step 9:- END
```

code:

```
#include <iostream>
using namespace std;

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;
        }
    }

    return -1;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);

    int key = 30;

    int result = linearSearch(arr, n, key);

    if (result != -1)
        cout << "Element found at index " << result;
    else
        cout << "Element not found";

    return 0;
}
```

Output:

```
Output
Element found at index 2

=== Code Execution Successful ===
```

Time complexity:

① Best case:-
The element is present at the first position.
Ex:- A = 10 20 30 40 50
Key = 10
 $T(n) = 1$
 $T(n) = O(1)$

② Average case:-
Suppose the element is somewhere in the middle
Ex:- A = 10 20 30 40 50
Key = 30
 $T(n) = n/2$
 $T(n) = O(n)$

③ worst case:-
Element in last position (or) not element present
A = 10 20 30 40 50
Key = 50
 $T(n) = n$
 $T(n) = O(n)$

2) Binary search

Input:- Sorted array "A" with 'n' elements and search element 'key'.
Output:- position of key if found, otherwise display "element not found".

Step 1:- Input n
 Input sorted array A [0 ... n-1]
 Input key

Step 2:- low $\leftarrow$ 0
 high $\leftarrow$ n-1

Step 3:- while low $\leq$ high
 mid $\leftarrow$ (low+high)/2

Step 4:- IF A[mid] = key then
 output mid
 stop

Step 5:- else if A[mid] < key then
 low $\leftarrow$ mid+1

Step 6:- Else
 high $\leftarrow$ mid-1

Step 7:- END IF
 END while

Step 8:- Output "Element not found"

Step 9:- END

code:

```
#include <iostream>
using namespace std;
int binarySearch(int arr[], int n, int key) {
    int low = 0;
    int high = n - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == key) {
            return mid;
        }
        else if (arr[mid] < key) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }

    return -1;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);

    int key = 40;

    int result = binarySearch(arr, n, key);

    if (result != -1)
        cout << "Element found at index " << result;
    else
        cout << "Element not found";

    return 0;
}
```

Output:

```
Output
Element found at index 3

=== Code Execution Successful ===
```

Time complexity:

① Best case:-
The element is found at the middle position in the first comparison.
Ex:- $A = 10 \ 20 \ 30 \ 40 \ 50$
key = 30
 $A[\text{mid}] = \text{key}$
 $T(n) = 1$
 $T(n) = O(1)$

② Average case:-
On average, the search also eliminates approximately half of the remaining elements at each step.
 $T(n) \approx \log_2 n$
 $T(n) = O(\log n)$

③ worst case:-
This is where Binary search becomes interesting
 $n \rightarrow n/2 \rightarrow n/4 \dots \dots 1 \rightarrow$
 $\frac{n}{2^k} = 1$
 $n = 2^k$
 $k = \log_2 n$
 $T(n) = \log n$
 $T(n) = O(\log n)$

Conclusion:

In this practical, we implemented Linear Search and Binary Search and understood their working and time complexities. Linear Search checks elements one by one, while Binary Search repeatedly divides a sorted array into halves. Thus, Binary Search is more efficient for large sorted datasets.